SIMATS SIMATS

Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	**192565031**
Student Name	**JAYA HARINI B R**

Instructions: Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result, and interpret what each result means for the platform. Assume the platform's servers execute 10^8 basic operations per second.

IT APPLICATION-BASED PROBLEM

Scenario – Scalability Audit of "ShopSphere", an Online Shopping Platform: ShopSphere is a growing e-commerce platform whose product catalogue currently holds n = 100,000 products and is expected to reach n = 1,000,000 within a year. Before the expansion, the CTO has asked you, as the platform's algorithm engineer, to audit five back-end modules, predict how each will behave as n grows, and recommend which modules must be redesigned. The modules are described below.

Module M1 – Duplicate-Listing Detector: Sellers sometimes upload the same product twice. Every night, the following routine compares every product with every later product.

```
FOR i ← 0 TO n − 2 DO
   FOR j ← i + 1 TO n − 1 DO
      IF Product[i].title = Product[j].title AND Product[i].seller =
Product[j].seller THEN
         MarkDuplicate(j)
      END IF
   END FOR
END FOR
```

Module M2 – API Rate Limiter: The public API audits request windows of doubling size using the routine below. A team member claims that it is Θ(n log n) because the outer loop runs about log n times and the inner loop runs up to n times.

```
i ← 1
WHILE i < n DO
   FOR j ← 0 TO i − 1 DO
      Audit(Request[j])          // basic operation
   END FOR
   i ← i × 2
END WHILE
```

Module M3 – Product Search: Two versions exist. Version S1 scans the unsorted catalogue sequentially until the product is found. Version S2 searches a sorted product index by comparing the key with the middle entry and discarding half the index after each comparison; its comparison count satisfies T(n) = T(n/2) + 1, T(1) = 1.

Module M4 – Recommendation Engine: The engine splits the customer–product data into four parts of size n/2 (because of replicated partitions) and merges the partial results in linear time: T(n) = 4T(n/2) + n. A proposed redesign removes the replication, giving T(n) = 2T(n/2) + n.

Module M5 – Sales Analytics Engine: The analytics engine splits the sales data into halves recursively and sorts the records by revenue at every level, giving T(n) = 2T(n/2) + n log n. Separately, profiling shows that the nightly report generator performs $f(n) = 3n^2 + 10n \log_2 n + 50$ operations.

(a) Understand the Problem: For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

(b) Analyse the Iterative Modules: For M1, derive the exact number of comparisons as a function of n and state the time and space complexity. For M2, derive the exact number of Audit operations, decide whether the team member's Θ(n log n) claim is correct and give the tight bound. Verify your M2 result for n = 1,024.

(c) Solve a Recurrence by Substitution: Solve the recurrence for search version S2 using backward substitution, showing every intermediate step, and derive the exact closed form. State the time complexity and the space complexity of both an iterative and a recursive implementation.

(d) Apply the Master Theorem: For M4, apply the Master Theorem to both the current design and the proposed redesign, clearly identifying a, b, f(n), n^(log_b a) and the case used, and explain what happens to the running time when the data doubles. For M5, determine whether any of the three standard cases applies to T(n) = 2T(n/2) + n log n; if not, derive the complexity using the extended Master Theorem or a recursion tree.

(e) Analyse Best, Average and Worst Cases: For search versions S1 and S2, identify the input condition that produces the best, average and worst case and give the number of comparisons in each. For S1, derive the average number of comparisons when the probability that the searched product exists is p, and evaluate it for n = 1,000,000 and p = 0.5.

(f) Prove an Asymptotic Bound: Using the formal definitions, prove that the report generator's cost $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$ by finding constants c_1, c_2 and n_0, and show that f(n) is not O(n).

(g) Scalability Decision: Estimate the number of basic operations and the running time of M1, M2, S2, M4 (both designs) and M5 for n = 1,000,000. Identify the modules that will fail to meet a nightly processing window of 10 minutes, and recommend, with justification, which modules must be redesigned and how.

Rubrics – Application-Based Question

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Code of Conduct

I JAYA HARINI B R (192565031) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: JAYA HARINI B R Date:13-10-2026

ANSWERS

(a) Understand the Problem

The input size n represents the number of products or records handled by the platform. For each module, the basic operation and its growth rate are identified below.

Module	Dominant operation	Why it dominates	Growth
M1 – Duplicate detector	Compare product title and seller	The routine examines every pair of products.	$\Theta(n^2)$
M2 – API rate limiter	Execute Audit(Request[j])	The inner-loop sizes form 1 + 2 + 4 + …, a geometric series.	$\Theta(n)$
M3 S1 – Sequential search	Compare the key with a product	It may inspect every catalogue entry.	$\Theta(n)$ worst case
M3 S2 – Binary search	Compare the key with the middle entry	The remaining search interval is halved each time.	$\Theta(\log n)$
M4 – Current recommendation engine	Process recursive subproblems and merge	Four recursive calls are made on inputs of size $n/2$.	$\Theta(n^2)$
M4 – Proposed redesign	Process recursive subproblems and merge	Two recursive calls are made on inputs of size $n/2$.	$\Theta(n \log n)$
M5 – Sales analytics	Process and sort data at recursive levels	The recurrence adds n log n work at each level.	$\Theta(n \log^2 n)$
Report generator	Execute operations in $3n^2 + 10n \log_2 n + 50$	The quadratic term dominates as n grows.	$\Theta(n^2)$

Interpretation: Quadratic routines become costly as the catalogue grows. Linear and logarithmic routines usually scale better, while $n \log^2 n$ grows more slowly than n^2 for large n.

(b) Analyse the Iterative Modules

Duplicate-Listing Detector

For i = 0, the inner loop makes n − 1 comparisons. For i = 1, it makes n − 2 comparisons. This continues until the final comparison.

$$\mathbf{C(n) = (n - 1) + (n - 2) + \ldots + 2 + 1}$$

Using the sum of the first n − 1 positive integers:

$$\mathbf{C(n) = n(n - 1) / 2}$$

Expanding gives $C(n) = \frac{1}{2}n^2 - \frac{1}{2}n$. The n^2 term dominates, so the time complexity is $\Theta(n^2)$. The routine uses only a fixed number of loop variables, so its auxiliary space complexity is $\Theta(1)$, excluding the catalogue and any storage used to record duplicates.

Time: $\Theta(n^2)$ Auxiliary space: $\Theta(1)$

Conclusion: The pairwise comparison approach should be redesigned before the catalogue reaches one million products.

API Rate Limiter

The variable i starts at 1 and doubles after each outer-loop iteration. The values are 1, 2, 4, 8, …, up to the largest power of two less than n.

For integer $n > 1$, the number of outer-loop iterations is $k = \lceil \log_2 n \rceil$. Each iteration performs i Audit operations, so:

$$A(n) = 1 + 2 + 4 + \ldots + 2^{(k-1)}$$

$$A(n) = (2^k - 1) / (2 - 1) = 2^k - 1$$

$$A(n) = 2^{\lceil \log_2 n \rceil} - 1$$

Because $n \leq 2^{\lceil \log_2 n \rceil} < 2n$, the total is bounded above and below by constant multiples of n. Therefore $A(n) = \Theta(n)$. The team member's $\Theta(n \log n)$ claim is incorrect: the inner loop does not perform n operations on every outer iteration; its sizes form a geometric series.

Verification for n = 1,024:

$$A(1{,}024) = 1 + 2 + 4 + \ldots + 512 = 2^{10} - 1 = 1{,}023 \text{ operations}$$

For n = 1, the loop does not run, so $A(1) = 0$.

(c) Solve the S2 Recurrence by Backward Substitution

The given recurrence is $T(n) = T(n/2) + 1$, with $T(1) = 1$.

First substitution:

$$T(n) = T(n/2) + 1$$

Substitute the recurrence for $T(n/2)$:

$$T(n) = T(n/4) + 1 + 1 = T(n/4) + 2$$

Substitute again:

$$T(n) = T(n/8) + 3$$

After k substitutions:

$$T(n) = T(n/2^k) + k$$

The recursion reaches its base case when $n/2^k = 1$. Hence $2^k = n$ and $k = \log_2 n$, assuming n is a power of two.

$$T(n) = T(1) + \log_2 n = 1 + \log_2 n$$

Therefore the closed form for powers of two is $T(n) = \log_2 n + 1$, and the time complexity is $\Theta(\log n)$. For arbitrary input sizes, rounding depends on the exact midpoint and interval convention, but the logarithmic bound remains.

Implementation	Auxiliary space	Reason
Iterative binary search	$\Theta(1)$	Only a fixed number of index and key variables are used.
Recursive binary search	$\Theta(\log n)$	The recursive call stack grows with the number of halvings.

Conclusion: Binary search is much more efficient than a sequential scan on a large, sorted product index.

(d) Apply the Master Theorem

Current design: T(n) = 4T(n/2) + n

Compare the recurrence with T(n) = aT(n/b) + f(n). Here a = 4, b = 2, and f(n) = n.

$$\mathbf{n^{(\log_b a)} = n^{(\log_2 4)} = n^2}$$

Since f(n) = n = O(n^(2−1)), f(n) is polynomially smaller than n^2. Master Theorem Case 1 applies.

$$\mathbf{T(n) = \Theta(n^2)}$$

The four recursive subproblems create substantial repeated work, so the current design scales poorly.

Proposed redesign: T(n) = 2T(n/2) + n

Here a = 2, b = 2, and f(n) = n.

$$\mathbf{n^{(\log_b a)} = n^{(\log_2 2)} = n}$$

Since $f(n) = \Theta(n^{(\log_b a)})$, Master Theorem Case 2 applies.

$$\mathbf{T(n) = \Theta(n \log n)}$$

When the input doubles, the current quadratic design takes approximately four times as much work. For the redesigned n log n algorithm, the ratio is approximately $2(\log_2 n + 1)/\log_2 n$, which approaches 2 as n becomes large. Removing replication therefore improves scalability.

Sales analytics: $T(n) = 2T(n/2) + n \log_2 n$

Here a = 2, b = 2, and n^(log_b a) = n. The non-recursive work is f(n) = n log n, which is larger than n by a logarithmic factor. The three basic Master Theorem cases do not directly give the result, so use the extended Master Theorem.

For $f(n) = \Theta(n^{(\log_b a)} \log^k n)$, the extended result is $T(n) = \Theta(n^{(\log_b a)} \log^{(k+1)} n)$. In this recurrence k = 1.

$$\mathbf{T(n) = \Theta(n \log^2 n)}$$

Conclusion: M5 grows faster than n log n but slower than n^2 for sufficiently large inputs.

(e) Best, Average and Worst Search Cases

Sequential search

Case	Input condition	Comparisons
Best	The target is the first product.	1
Average, target present	The target is equally likely to be at any position.	$(n + 1)/2$
Worst, target present	The target is the last product.	n
Worst, target absent	The target is not in the catalogue.	n

Let p be the probability that the target exists. If present, the expected number of comparisons is $(n + 1)/2$. If absent, all n entries are checked. By the law of total expectation:

$$\mathbf{E[C] = p((n + 1)/2) + (1 - p)n}$$

$$\mathbf{E[C] = n(1 - p/2) + p/2}$$

For $n = 1{,}000{,}000$ and $p = 0.5$:

$$\mathbf{E[C] = 1{,}000{,}000(1 - 0.5/2) + 0.5/2}$$

$$\mathbf{E[C] = 750{,}000.25 \text{ comparisons}}$$

Thus the expected number is approximately 750,000 comparisons.

Binary search

Case	Input condition	Comparison count
Best	The target is the middle entry on the first comparison.	1
Average	The target is found after a typical sequence of halvings.	$\Theta(\log n)$
Worst	The target is at the deepest level or is absent.	$\Theta(\log n)$

For $n = 1{,}000{,}000$, $\log_2 n \approx 19.93$, so binary search needs about 20–21 comparisons in the worst case, depending on the implementation. S2 is preferable for a large catalogue when a sorted index is maintained.

(f) Prove the Report Generator is $\Theta(n^2)$ and not $O(n)$

The cost function is $f(n) = 3n^2 + 10n \log_2 n + 50$.

Definition: $f(n)$ is $\Theta(n^2)$ if there are positive constants c_1, c_2, and n_0 such that $0 \le c_1 n^2 \le f(n) \le c_2 n^2$ for every $n \ge n_0$.

Lower bound: for $n \ge 1$, all terms are non-negative, so $f(n) \ge 3n^2$. Choose $c_1 = 3$.

Upper bound: for $n \ge 64$, $\log_2 n \le n/10$, so $10n \log_2 n \le n^2$. Also, $50 \le n^2$. Therefore:

$$\mathbf{f(n) = 3n^2 + 10n \log_2 n + 50 \le 3n^2 + n^2 + n^2 = 5n^2}$$

Choose $c_2 = 5$ and $n_0 = 64$. For every $n \ge 64$:

$$\mathbf{3n^2 \le f(n) \le 5n^2}$$

Thus, by definition, $f(n) = \Theta(n^2)$.

To prove f(n) is not O(n), suppose a constant $c > 0$ could satisfy $f(n) \leq cn$ for all sufficiently large n. But $f(n) \geq 3n^2$, so dividing by n gives $f(n)/n \geq 3n$. As n increases, 3n becomes arbitrarily large, so no fixed c can bound f(n)/n. This contradicts the definition of O(n).

Therefore, $f(n) \notin O(n)$.

(g) Scalability Decision

At n = 1,000,000, the server's nominal capacity for a 10-minute window is:

10^8 operations/second × 600 seconds = 60,000,000,000 operations

The following figures distinguish exact loop counts from illustrative recurrence-based estimates. M1 and M2 counts follow directly from their loops. S2 is per search. For M4 and M5, exact operation counts cannot be determined from the recurrences alone without base-case costs and implementation constants; the estimates below use simplified unit-cost models.

Module	**Operations / work estimate at n = 1,000,000**	**Approx. time at 10^8 ops/s**	**Assessment**
M1 – Duplicate detector	$n(n-1)/2$ = 499,999,500,000 comparisons	4,999.995 s ≈ 83.33 min	Fails 10-minute window
M2 – API rate limiter	2^20 − 1 = 1,048,575 Audit operations	0.0105 s	Meets under stated model
S2 – Binary search	About 20–21 comparisons per search	About 0.20–0.21 microseconds per search	Meets per individual search
M4 – Current design	$\Theta(n^2)$; dominant-term illustration ≈ 10^12 units	Illustration ≈ 10,000 s ≈ 166.67 min	Quadratic design is unsuitable
M4 – Redesigned	$n \log_2 n$ ≈ 19,931,569 units in a simplified model	Illustration ≈ 0.199 s	Scales much better
M5 – Sales analytics	Recursion-tree illustration $nL(L+1)/2$, $L = \log_2 n \approx 19.93$; about 2.1×10^8 units	Illustration ≈ 2.1 s	Likely feasible in simplified model; benchmark

Changes to be made

1. M1: Replace pairwise comparison with a hash table keyed by seller and normalized product title. Expected time is $\Theta(n)$, with $\Theta(n)$ auxiliary space. A sorting-based alternative takes $\Theta(n \log n)$ time.

2. M4: Remove unnecessary replication. The recurrence changes from $4T(n/2) + n = \Theta(n^2)$ to $2T(n/2) + n = \Theta(n \log n)$.

3. M2 and S2: Retain the algorithms, while testing the actual request volume and keeping the binary-search index sorted.

4. M5: It has $\Theta(n \log^2 n)$ growth, which is below quadratic growth. Retain provisionally and benchmark using production-sized data and the full nightly workload.

Final decision: Prioritize redesigning M1 and the current M4 because their quadratic growth creates the greatest scalability risk. Validate all estimated times with implementation benchmarks before making a production guarantee.